

Handling Multiple Browsers Using Playwright

Objective

The objective of this task is to learn how Playwright supports multiple browsers and how the same test can be executed on different browser engines such as Chromium, Firefox, and WebKit

Introduction

During automation testing, it is important to verify that an application works properly in different browsers. Users may access the application using Chrome, Firefox, Safari, or other browsers. A feature that works correctly in one browser may not behave the same way in another browser.

Playwright provides built-in support for multiple browsers, making cross-browser testing easier. In this task, I explored how to execute a Playwright test in Chromium, Firefox, and WebKit and verify that the application behaves consistently across all browsers.

Browsers Supported by Playwright

Playwright supports the following browser engines:

- Chromium
- Firefox
- WebKit

These browser engines allow us to perform cross-browser testing using the same automation script.

Task Scenario

For this task, the following steps were performed:

1. Open the browser.
2. Navigate to the Playwright website.
3. Verify that the page loads successfully.
4. Capture the page title.
5. Take a screenshot.

6. Close the browser.
7. Execute the same test in multiple browsers.

Playwright Configuration

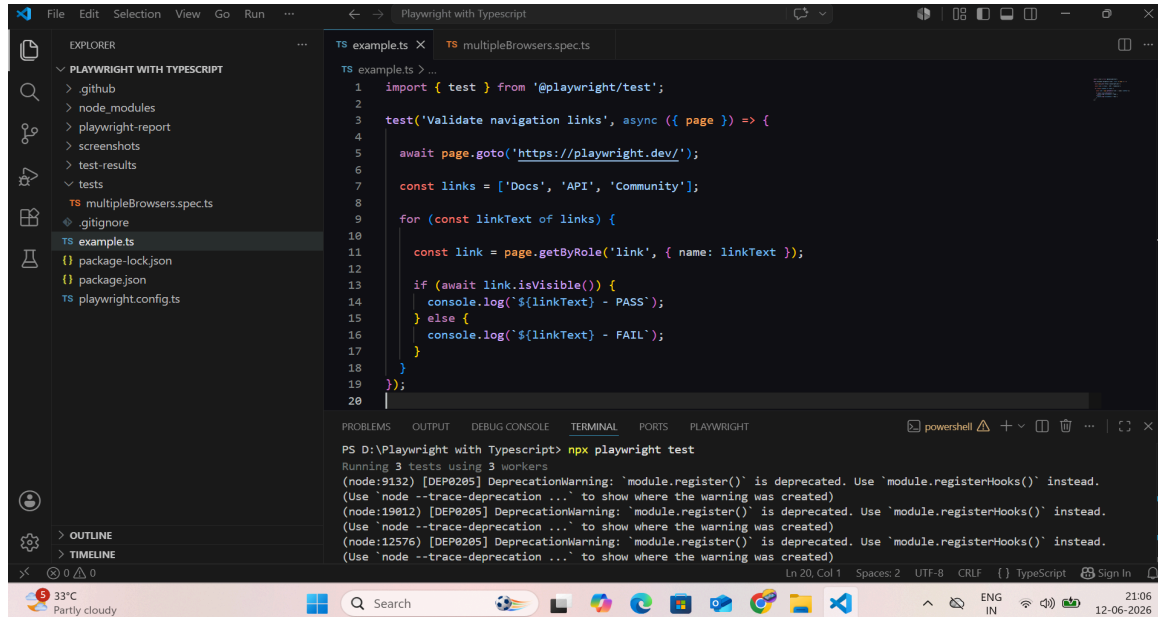
Playwright allows browser configuration through the project settings. Separate projects can be created for Chromium, Firefox, and WebKit so that the same test runs automatically in all browsers.

Example:

```
projects: [  
  
  {  
  
    name: 'chromium',  
  
    use: { browserName: 'chromium' },  
  
  },  
  
  {  
  
    name: 'firefox',  
  
    use: { browserName: 'firefox' },  
  
  },  
  
  {  
  
    name: 'webkit',  
  
    use: { browserName: 'webkit' },  
  
  },  
  
],
```

Automation Script

The following script was used to validate the application and capture screenshots in each browser.



The screenshot shows a Visual Studio Code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project named 'PLAYWRIGHT WITH TYPESCRIPT' with files like 'package-lock.json', 'package.json', and 'playwright.config.ts'. The code editor shows a file named 'multipleBrowsers.spec.ts' with the following TypeScript code:

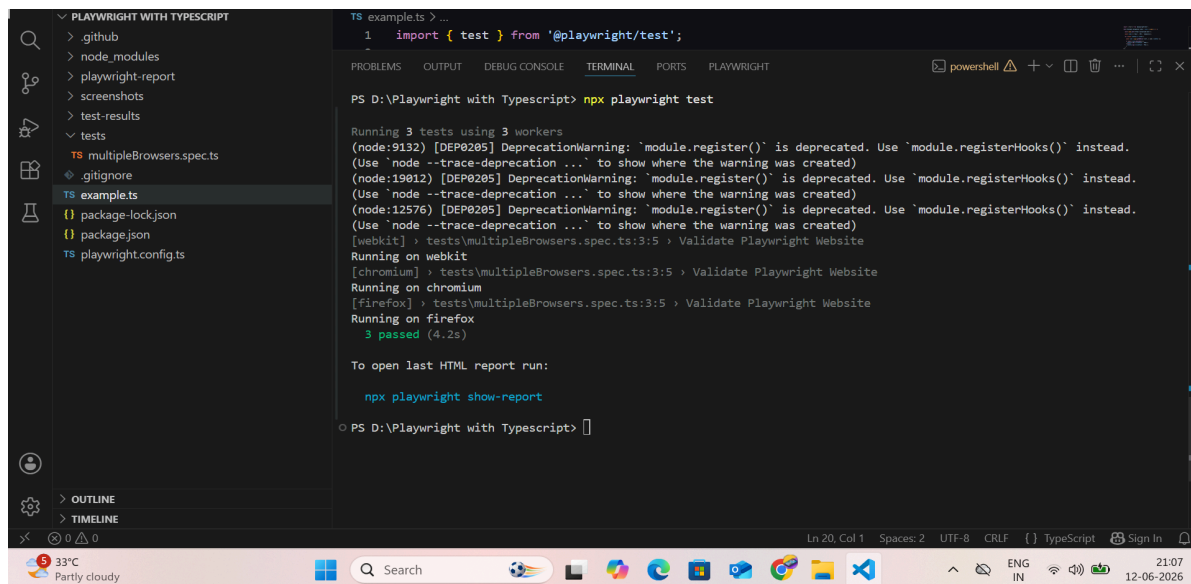
```
1 import { test } from '@playwright/test';
2
3 test('Validate navigation links', async ({ page }) => {
4
5   await page.goto('https://playwright.dev/');
6
7   const links = ['Docs', 'API', 'Community'];
8
9   for (const linkText of links) {
10
11     const link = page.getByRole('link', { name: linkText });
12
13     if (await link.isVisible()) {
14       console.log(`${linkText} - PASS`);
15     } else {
16       console.log(`${linkText} - FAIL`);
17     }
18   }
19 });
```

The bottom panel shows the 'TERMINAL' tab with the following output:

```
PS D:\Playwright with Typescript> npx playwright test
Running 3 tests using 3 workers
(node:9132) [DEP0205] DeprecationWarning: 'module.register()' is deprecated. Use 'module.registerHooks()' instead.
(Use 'node --trace-deprecation ...' to show where the warning was created)
(node:19012) [DEP0205] DeprecationWarning: 'module.register()' is deprecated. Use 'module.registerHooks()' instead.
(Use 'node --trace-deprecation ...' to show where the warning was created)
(node:12576) [DEP0205] DeprecationWarning: 'module.register()' is deprecated. Use 'module.registerHooks()' instead.
(Use 'node --trace-deprecation ...' to show where the warning was created)
```

Script for Multi Browser Testing

Output



The screenshot shows the same Visual Studio Code editor as before, but now the 'TERMINAL' tab displays the output of the test script. The output shows that the test passed and provides instructions on how to view the report.

```
PS D:\Playwright with Typescript> npx playwright test
Running 3 tests using 3 workers
(node:9132) [DEP0205] DeprecationWarning: 'module.register()' is deprecated. Use 'module.registerHooks()' instead.
(Use 'node --trace-deprecation ...' to show where the warning was created)
(node:19012) [DEP0205] DeprecationWarning: 'module.register()' is deprecated. Use 'module.registerHooks()' instead.
(Use 'node --trace-deprecation ...' to show where the warning was created)
(node:12576) [DEP0205] DeprecationWarning: 'module.register()' is deprecated. Use 'module.registerHooks()' instead.
(Use 'node --trace-deprecation ...' to show where the warning was created)
[webkit] > tests\multipleBrowsers.spec.ts:3:5 > Validate Playwright Website
Running on webkit
[chromium] > tests\multipleBrowsers.spec.ts:3:5 > Validate Playwright Website
Running on chromium
[firefox] > tests\multipleBrowsers.spec.ts:3:5 > Validate Playwright Website
Running on firefox
3 passed (4.2s)

To open last HTML report run:
npx playwright show-report

PS D:\Playwright with Typescript>
```

Output for Multiple Browser testing

Single Browser Execution

Playwright also allows execution on a specific browser when required.

To run only on Chromium:

```
npx playwright test --project=chromium
```

To run only on Firefox:

```
npx playwright test --project=firefox
```

To run only on WebKit:

```
npx playwright test --project=webkit
```

This is useful when debugging issues related to a particular browser.

Conclusion

Through this task, I learned how Playwright supports cross-browser testing using Chromium, Firefox, and WebKit. I was able to execute the same automation script across multiple browsers and verify the application behavior. I also explored how to run tests on a single browser whenever required for debugging or validation purposes.